Conflictos de fusión

GIT INTERMEDIO



George Boorman
Curriculum Manager, DataCamp

datacamp

Conflictos

- Conflicto
  - Incapacidad de resolver diferencias en el contenido de uno o varios archivos entre ramas
- Edita el mismo archivo en dos ramas
- Intenta fusionar
- Git no sabe qué versión mantener
- ¡Conflicto!

Versiones en conflicto de README.md

Rama `documentation`

```
# Contenido y uso

Este repo contiene código fuente
para el sitio web de DataCamp.

También contiene código fuente para un
Asistente de IA (sistema de recomendación)
que recibe indicaciones de estudiantes y
devuelve contenido sugerido
que podría interesarles.

Es solo para uso interno;
se prohíbe el acceso externo.
```

Rama `main`

```
# Contenido y uso

Este repo contiene código fuente
para el sitio web de DataCamp.

Es solo para uso interno;
se prohíbe el acceso externo.
```

Fusión

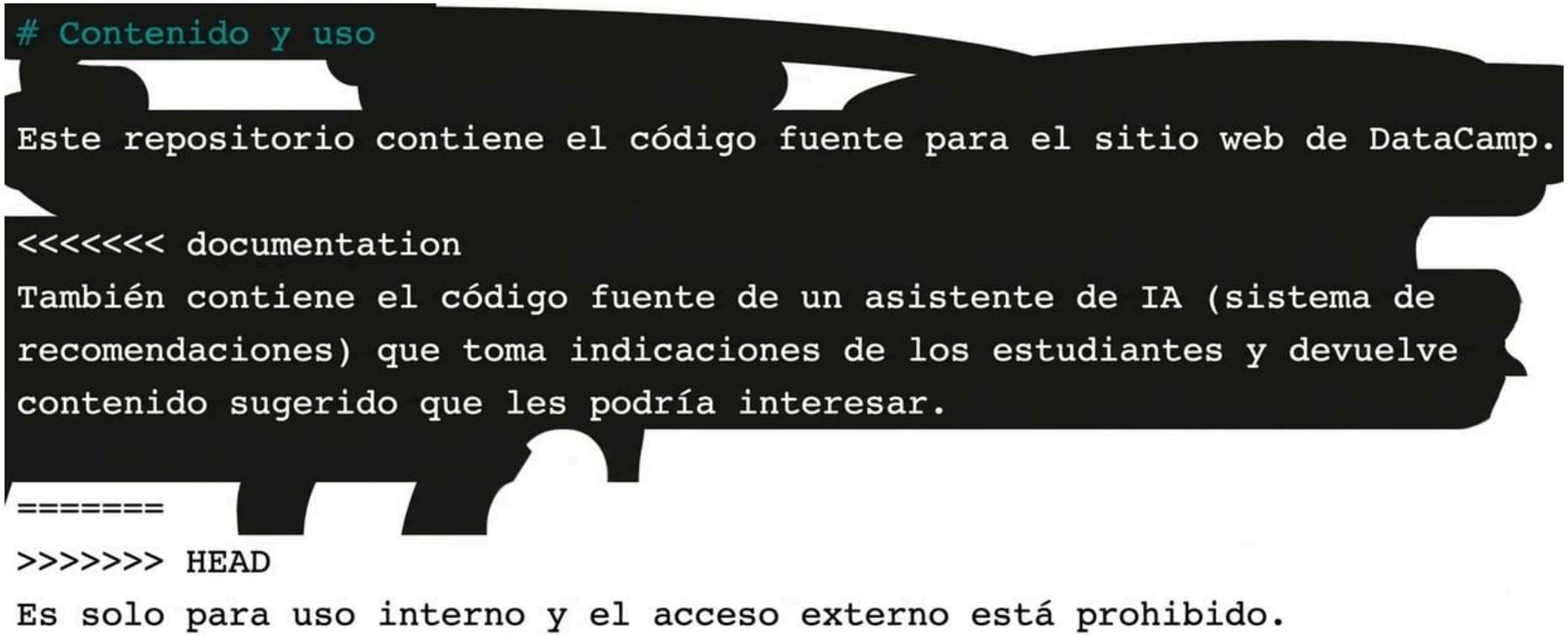
- Desde la rama `main`

```
git merge documentation
```

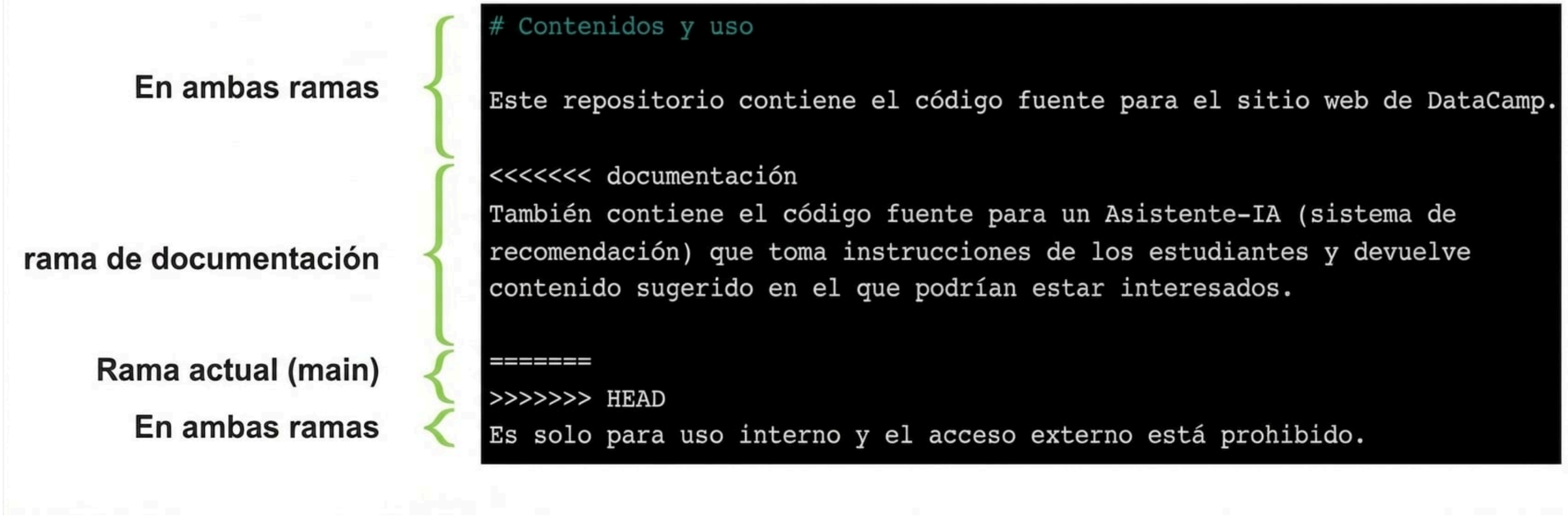
```
Auto-merging README.md
CONFLICT (add/add): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

Abrir el archivo

```
nano README.md
```



Sintaxis de conflictos en Git



Resolver el conflicto

```
# Contenido y uso

Este repositorio contiene el código fuente del sitio web de DataCamp.

<<<<<<< documentation
También contiene el código fuente para un asistente de IA (sistema de
recomendación) que recibe indicaciones de los estudiantes y devuelve
contenido sugerido que podría interesarles.

=======
>>>>>>> HEAD
Es solo para uso interno y está prohibido el acceso externo.
```

- Guardar: `Ctrl + O` (no `Ctrl + 0`), luego `Enter`
- Salir: `Ctrl + X`

Fusionar las ramas

- Fusionar ahora que el conflicto está resuelto

```
git add README.md
```

```
git commit -m "Resolving README.md conflict"
```

```
git merge documentation
```

```
Already up to date.
```

- Mejor prevenir que curar.

¡Vamos a practicar!

GIT INTERMEDIO

Introducción a los remotos

GIT INTERMEDIO



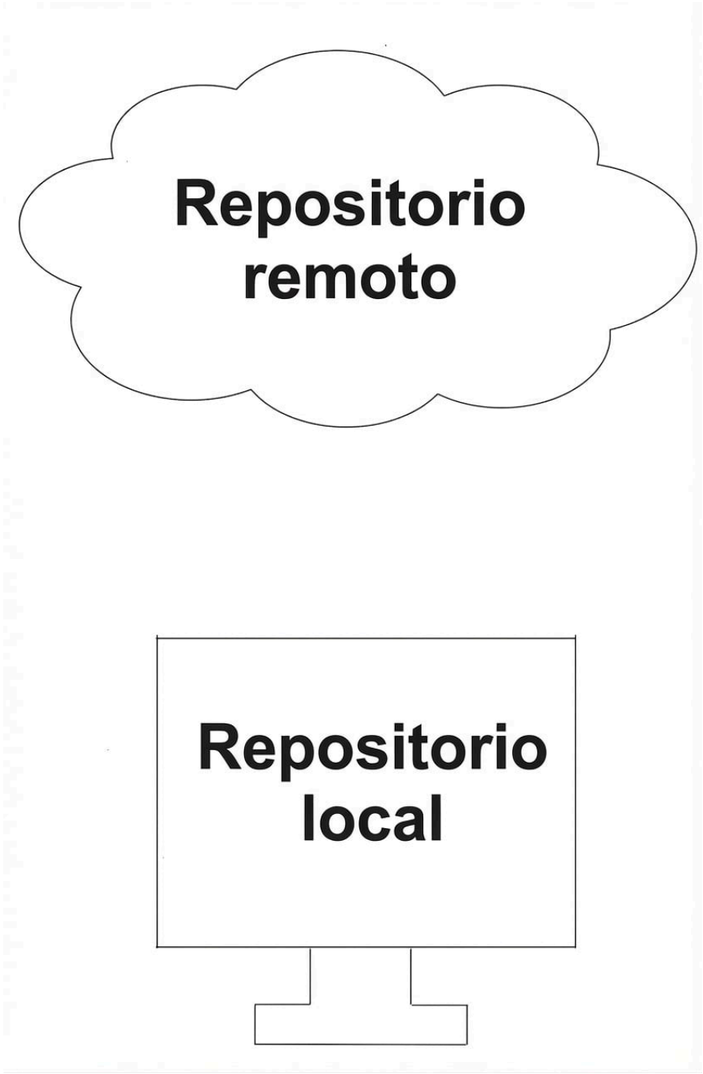
George Boorman
Curriculum Manager, DataCamp

datacamp

Repo local



Repo remoto



¿Por qué usar repos remotos?

Beneficios de los repos remotos

- Todo queda respaldado
- Colaboración desde cualquier lugar



Clonar un repo

- Copias del repo en el equipo local = remotos locales
- Crear copias = clonar

```
git clone path-to-project-repo
```

- Clonar un proyecto local

```
git clone /home/george/repo
```

- Clonar y nombrar un proyecto local

```
git clone /home/george/repo new_repo
```

Clonar un remoto

- Los repos remotos suelen estar en un servicio online
  - p. ej., GitHub, Bitbucket o GitLab
- Si tenemos cuenta:
  - Podemos clonar un repo remoto en el equipo local

```
git clone URL
```

```
git clone https://github.com/datacamp/project
```

Identificar un remoto

- Al clonar un repo
  - Git recuerda el origen
- Git guarda una etiqueta de remoto en la configuración del nuevo repo
- Lista todos los remotos del repo

```
git remote
```

```
datacamp
```

Obtener más información

- Obtén más información del/de los remoto(s)

```
git remote -v
```

```
datacamp     https://github.com/datacamp/project (fetch)
datacamp     https://github.com/datacamp/project (pull)
```

Crear un remoto

- Al clonar, Git nombra el remoto `origin` automáticamente

```
git remote add name URL
```

- Crea un remoto llamado `george`

```
git remote add george https://github.com/george_datacamp/repo
```

- Nombrar remotos ayuda al hacer merges

¡Vamos a practicar!

GIT INTERMEDIO

Obtener desde remotos

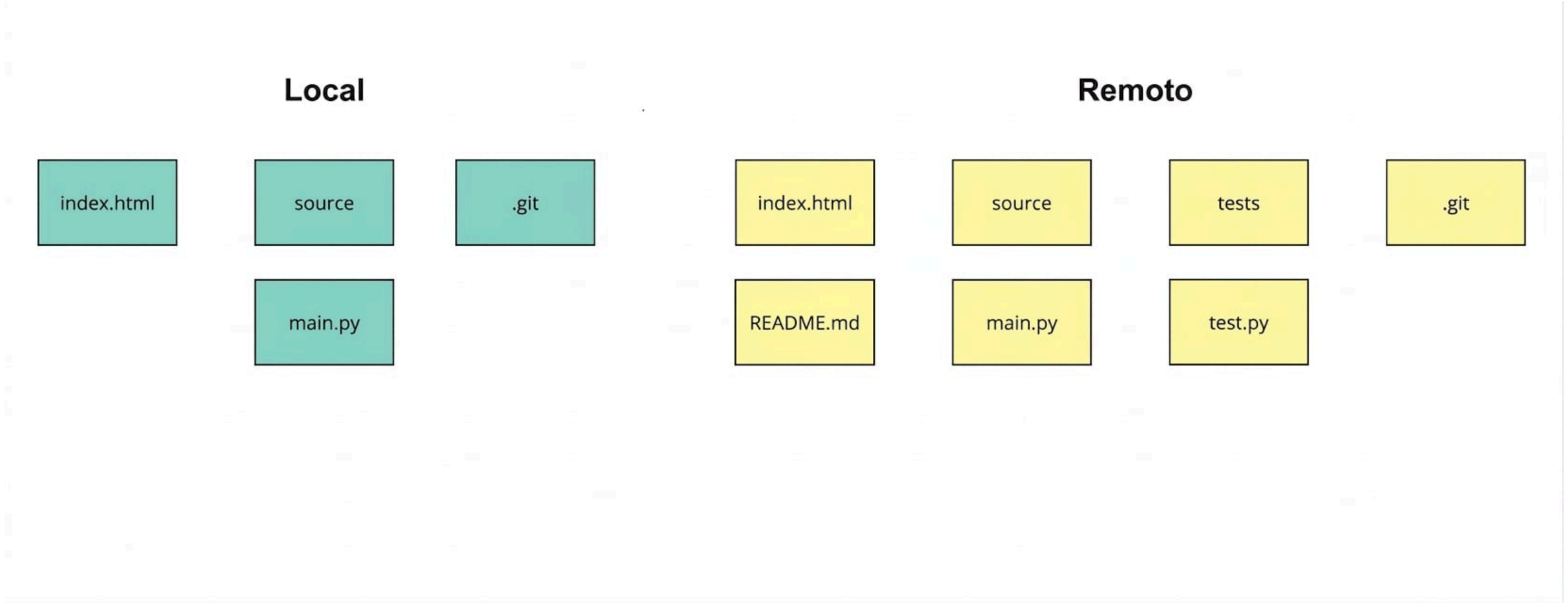
GIT INTERMEDIO



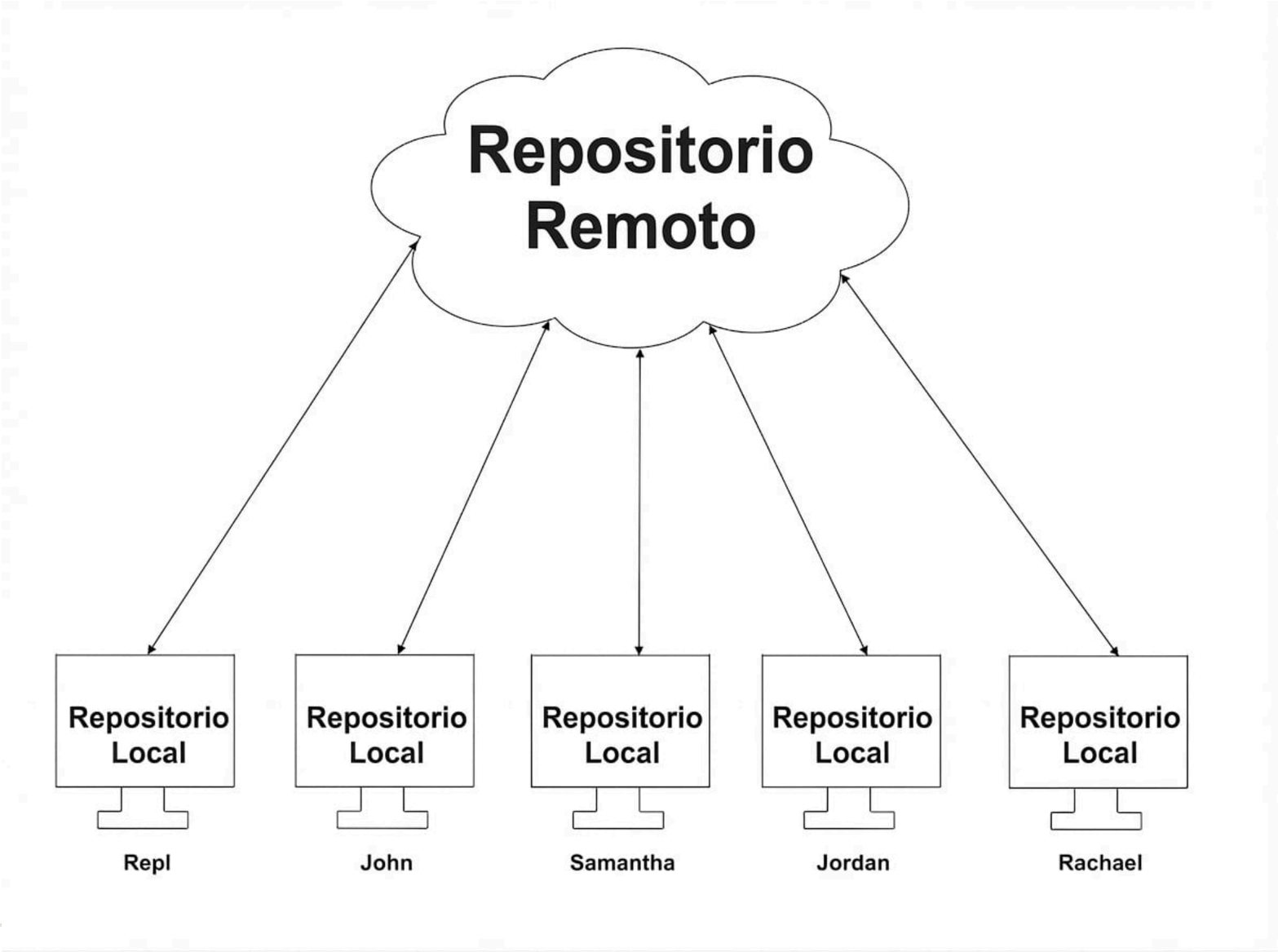
George Boorman
Curriculum Manager, DataCamp

datacamp

Remoto vs. local



Colaborar en proyectos Git



Traer desde un remoto

- Trae desde el remoto `origin`

```
git fetch origin
```

- Trae todas las ramas remotas
- Puede crear ramas locales nuevas si solo existían en el `remote`
- No fusiona el contenido del remoto en el repo local

Traer una rama remota

- Trae solo la rama `main` del remoto `origin`

```
git fetch origin main
```

```
From https://github.com/datacamp/project
 * branch                main     -> FETCH_HEAD
```

Sincronizar contenido

- Fusiona la rama predeterminada (`main`) del remoto `origin` en la rama local actual

```
git merge origin
```

```
Updating 9dcf4e5..887da2d
Fast-forward
 tests/tests.py | 13 ++++++++++++
 README.md      | 10 ++++++++++
 2 files changed, 23 insertions (+)
```

Obtener de un remoto

- Sincronizar local y remoto es un flujo común
- Git lo simplifica por nosotros
- Trae y fusiona la rama predeterminada (`main`) del remoto en la rama local actual

```
git pull origin
```

Obtener una rama remota

- Obtén la rama `dev` del remoto `origin`

```
git pull origin dev
```

- Aun así fusiona en la rama local donde estás

Salida de git pull

```
From https://github.com/datacamp/project
 * branch                dev     -> FETCH_HEAD
Updating 9dcf4e5..887da2d
Fast-forward
 tests/tests.py | 13 ++++++++++++
 README.md      | 10 ++++++++++
 2 files changed, 23 insertions (+)
```

Aviso importante

```
git pull origin
```

```
Updating 9dcf4e5..887da2d
error: Your local changes to the following files would be overwritten by merge:
        README.md
Please commit your changes or stash them before you merge.
Aborting
```

- Importante guardar localmente antes de obtener de un `remote`

¡Vamos a practicar!

GIT INTERMEDIO

Enviar a remotos

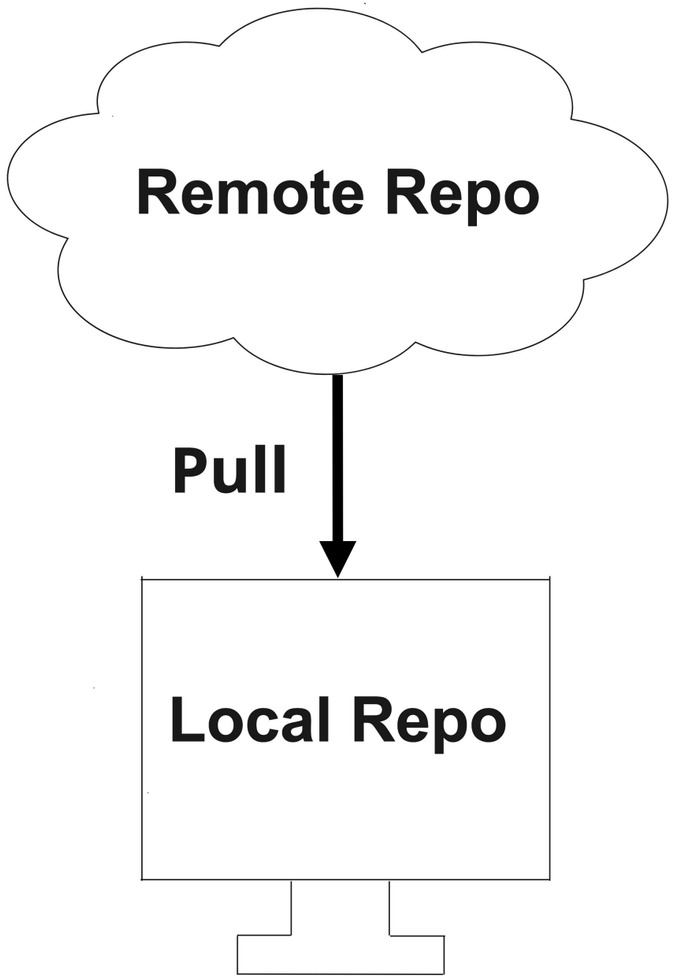
GIT INTERMEDIO



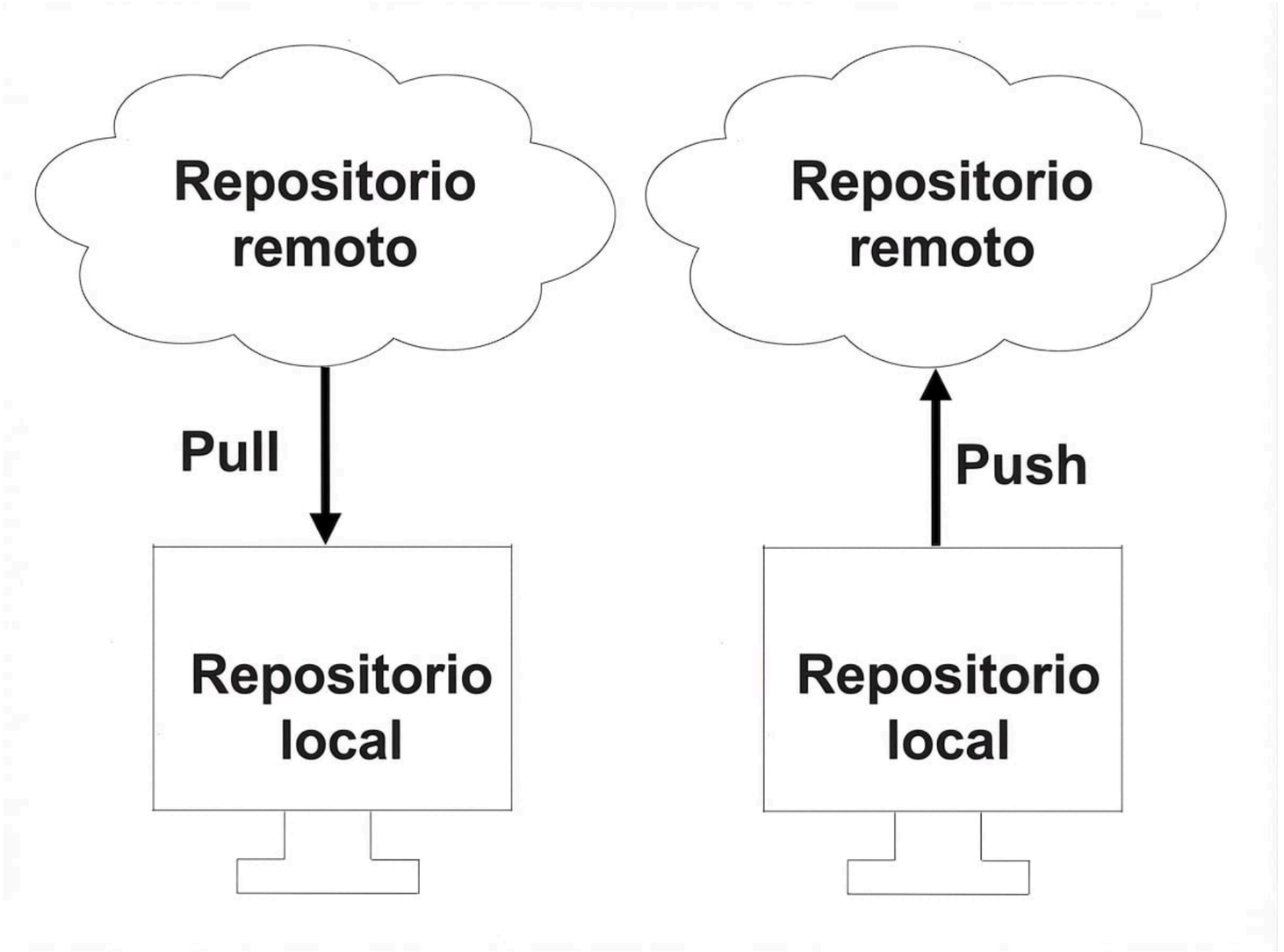
George Boorman
Curriculum Manager, DataCamp

datacamp

Hacer pull de un remoto



Empujar a un remoto



git push

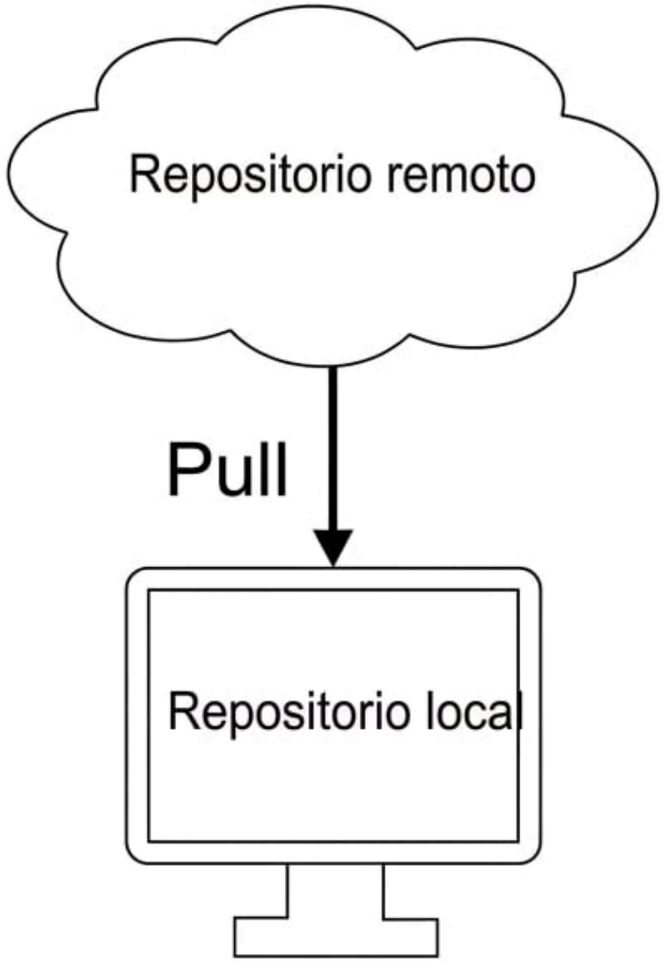
- ¡Guarda los cambios localmente primero!

```
git push remote local_branch
```

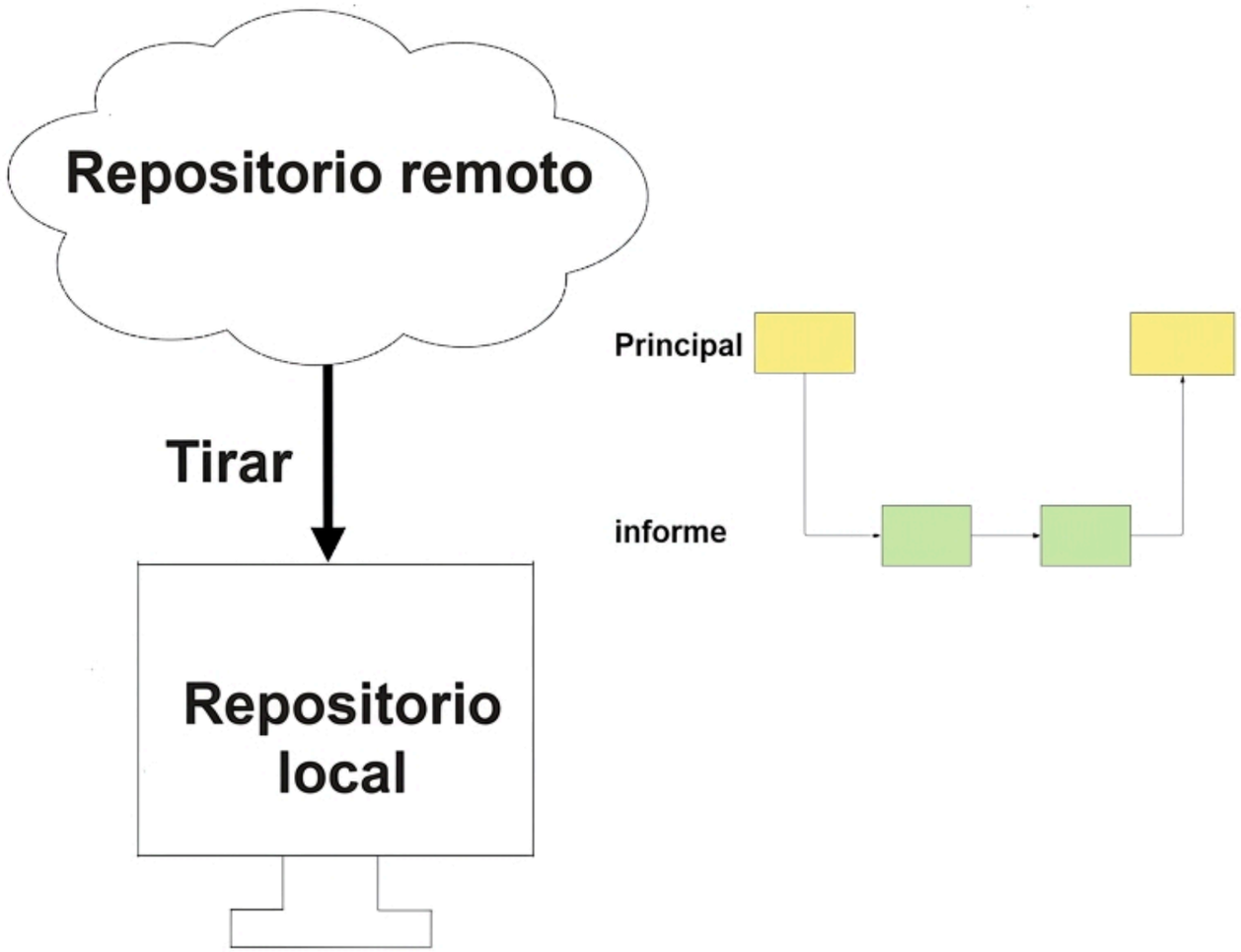
- Empuja _a_ `remote` **desde** `local_branch`
- Empuja cambios a `origin` desde `main` del repo local

```
git push origin main
```

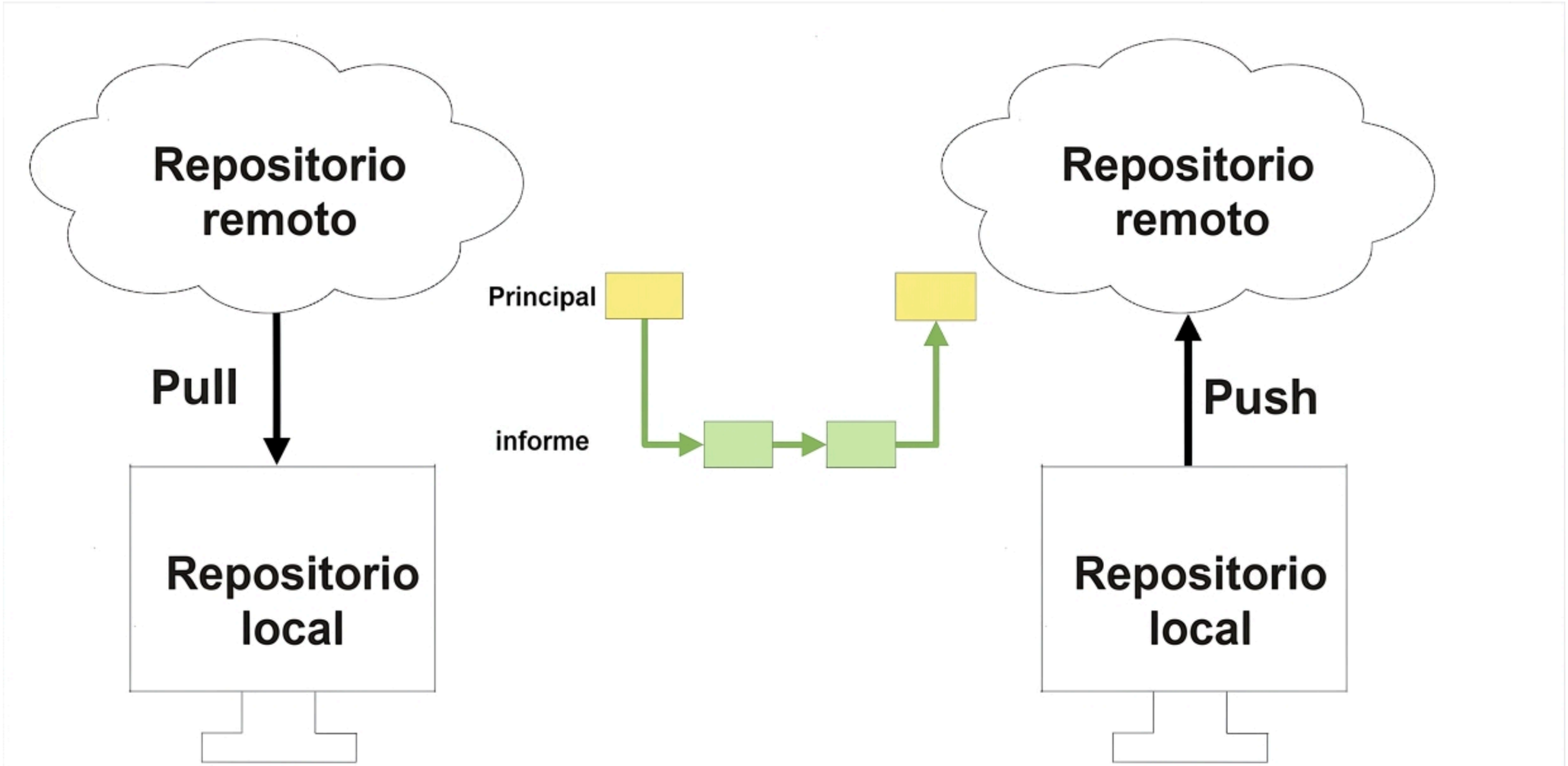
Flujo push/pull



Flujo push/pull



Flujo push/pull



Primero, push

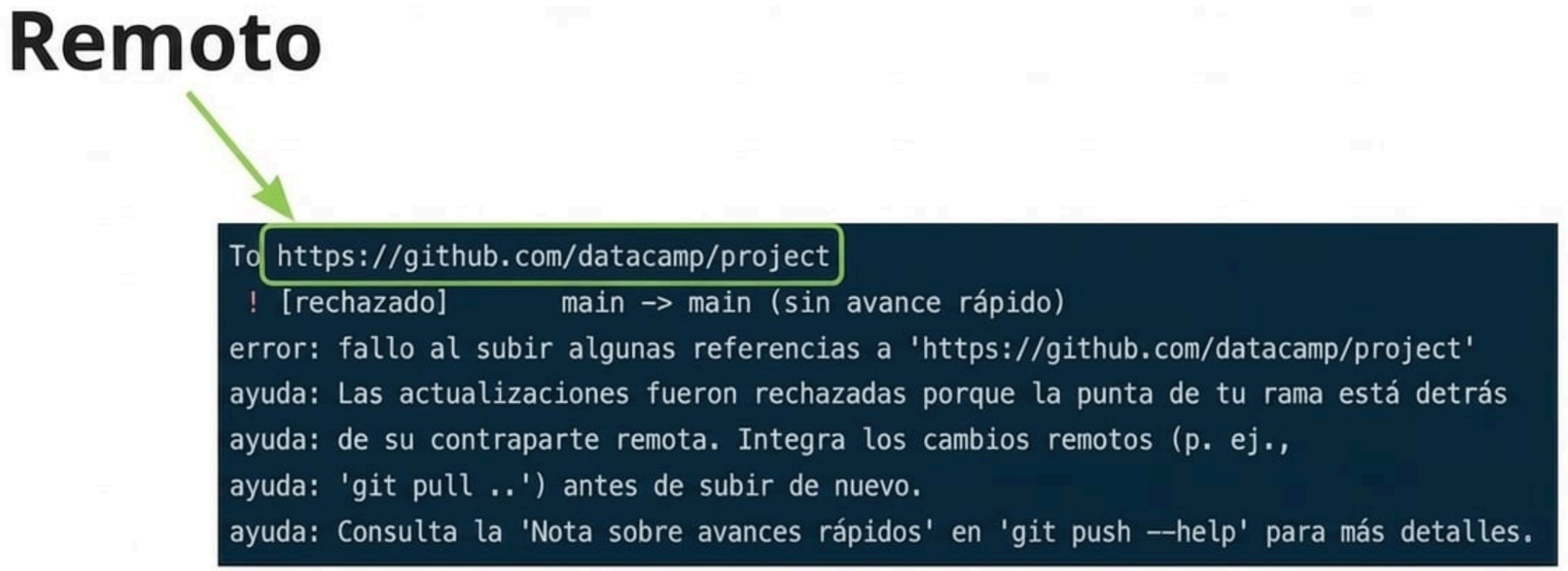
- Empujar `main` al remoto antes de hacer pull

```
git push origin main
```

Conflictos remoto/local

```
A https://github.com/datacamp/project
 ! [rechazado]          main -> main (no es avance rápido)
error: falló el envío de algunas referencias a 'https://github.com/datacamp/project'
ayuda: Las actualizaciones fueron rechazadas porque la punta de tu rama está detrás
ayuda: de su contraaparte remota. Integra los cambios remotos (p. ej.
ayuda: 'git pull ...') antes de volver a hacer push.
ayuda: Consulta la 'Nota sobre avances rápidos' en 'git push --help' para más detalles.
```

Conflictos remoto/local



Conflictos remoto/local

Resultado

```
To https://github.com/datacamp/project
 ! [rejected]        main -> main (non-fast-forward)
error: no se pudieron enviar algunas referencias a 'https://github.com/datacamp/project'
hint: Las actualizaciones fueron rechazadas porque la punta de tu rama está detrás
hint: de su contraparte remota. Integra los cambios remotos (p. ej.
hint: 'git pull ..') antes de volver a enviar.
hint: Consulta la 'Nota sobre los fast-forwards' en 'git push --help' para más detalles.
```

Conflictos remoto/local

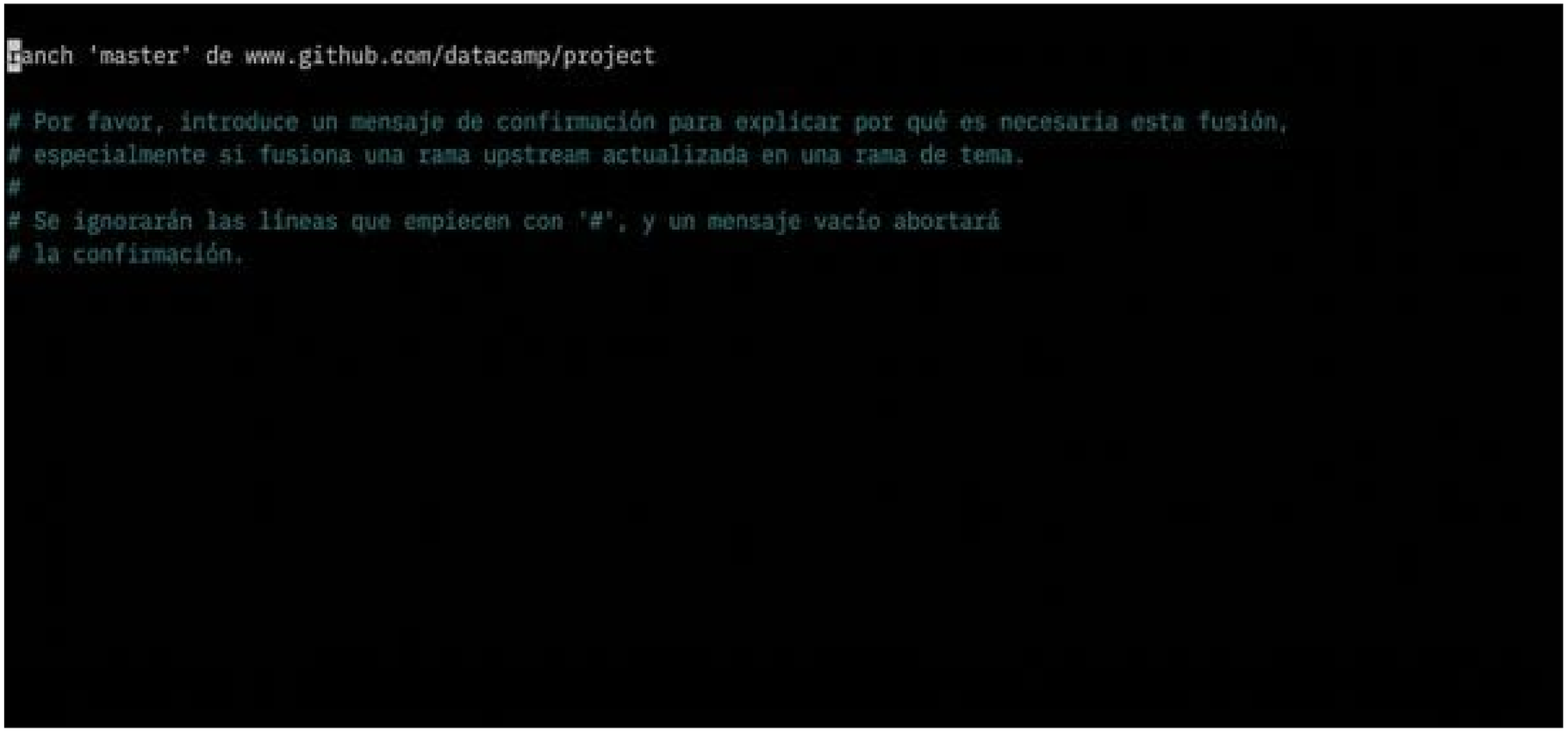
Causa(s) y sugerencia(s)

```
To https://github.com/datacamp/project
 ! [rejected]        main -> main (non-fast-forward)
error: falló al hacer push de algunas referencias a 'https://github.com/datacamp/project'
hint: Se rechazaron las actualizaciones porque la punta de tu rama está detrás
hint: de su contraparte remota. Integra los cambios remotos (p. ej.
hint: 'git pull ..') antes de volver a hacer push.
hint: Mira la 'Nota sobre fast-forwards' en 'git push --help' para más detalles.
```

Evitar un conflicto

- Haz pull del remoto primero

```
git pull origin main
```



Pull sin editar

```
git pull --no-edit origin main
```

- No recomendado, salvo que tengamos mucha confianza en el historial del proyecto.

Empujar una rama local nueva

- Trabajando en la rama `hotfix` local
- `hotfix` no existe en el remoto

Crear una rama remota nueva

- `hotfix` solo existe localmente

```
git push origin hotfix
```

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 349 bytes | 349.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'hotfix' on GitHub by visiting:
remote:      https://github.com/datacamp/project/pull/new/hotfix
remote:
To https://github.com/datacamp/project
 * [new branch]      hotfix -> hotfix
branch 'hotfix' set up to track 'origin/hotfix'.
```

¡Vamos a practicar!

GIT INTERMEDIO

¡Enhorabuena!

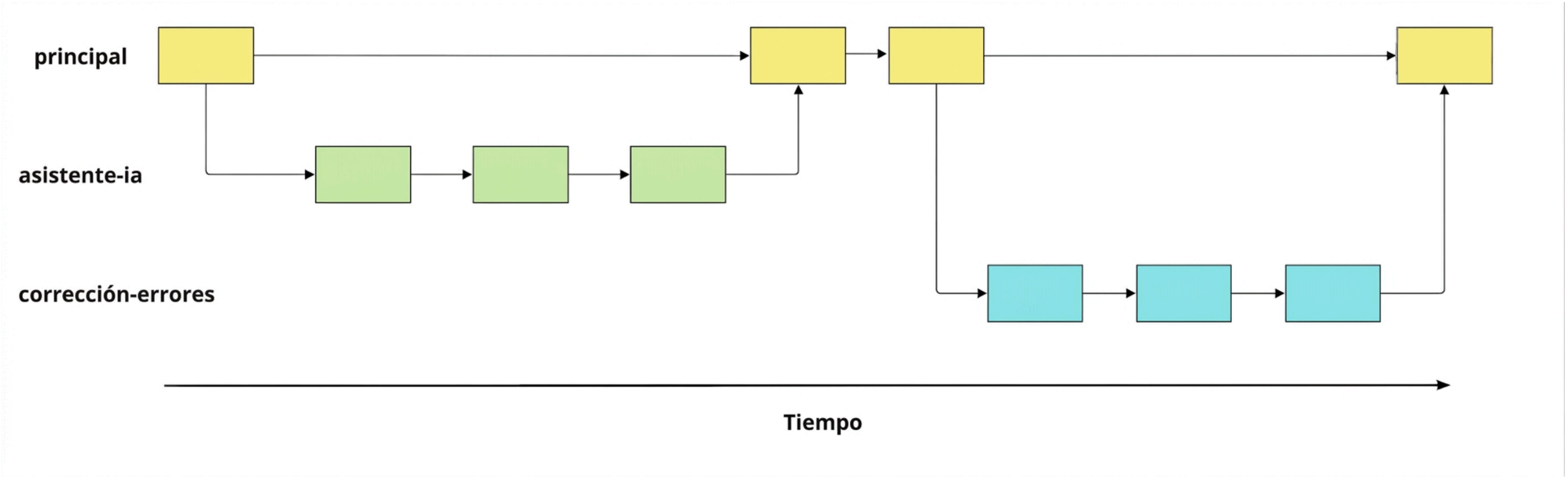
GIT INTERMEDIO



George Boorman
Curriculum Manager, DataCamp

datacamp

Ramas



Trabajar con ramas

- Ver todas las ramas

```
git branch
```

- Cambiar a una rama existente

```
git switch hotfix
```

- Crear y cambiar a una nueva rama

```
git switch -c hotfix
```

- Comparar dos ramas

```
git diff main hotfix
```

- Renombrar una rama

```
git branch -m hotfix bugfix
```

- Borrar una rama

```
git branch -d hotfix
```

Fusionar ramas

- Desde `main`, para fusionar `ai-assistant` en `main`:

```
git merge ai-assistant
```

- Desde otra rama: `git merge origen destino`

```
git merge ai-assistant main
```

- Resolver conflictos de fusión

Trabajar con remotos

- Clonar un repo remoto

```
git clone https://github.com/datacamp/project
```

- Ver información de todos los remotos

```
git remote -v
```

- Añadir un remoto nuevo

```
git remote add george https://github.com/george_datacamp/repo
```

Sincronizar repos locales y remotos

```
git fetch origin
```

```
git pull origin
```

```
git push origin documentation
```

¿Qué sigue?

- Conceptos de GitHub
- Tutorial de GitHub y Git
- ¡Empieza a usar Git en tus proyectos de software y datos!

¡Vamos a practicar!

GIT INTERMEDIO